

Phase 7: Sorted Array Operations and Rearrangement — Full Structured Notes

1. Big Idea of Phase 7

Phase 7 teaches three important Array ADT operations:

1. Insert a value into an already sorted array.
2. Check whether an array is sorted.
3. Rearrange negative and positive values.

In earlier phases, you learned how to:

- Store values using an Array ADT.
- Add values using append and insert.
- Delete values using shifting.
- Search using linear search and binary search.
- Reverse, shift, and rotate array elements.

Now Phase 7 focuses on arrays where **order matters**.

Simple idea:

Phase 7 = keep sorted arrays sorted, check sorted order, and rearrange values using two pointers.

2. Quick Review: Array ADT Structure

We continue using this structure:

```
struct Array {  
    int A[20];  
    int size;  
    int length;  
};
```

Meaning:

Part	Meaning
A	The actual array storage

Part	Meaning
<code>size</code>	Total capacity of the array
<code>length</code>	Number of valid elements currently stored

Example:

```
struct Array arr = {{4, 8, 13, 16, 20}, 20, 5};
```

This means:

```
A      = [4, 8, 13, 16, 20, _, _, ...]
size   = 20
length = 5
```

Only these indexes are valid:

```
0, 1, 2, 3, 4
```

The last valid index is:

```
length - 1
```

So here:

```
length - 1 = 5 - 1 = 4
```

Important rule:

Use `length` when working with real values. Use `size` only when checking capacity.

3. What Phase 7 Operations Do

Phase 7 has three main operations:

Operation	Purpose
<code>insertSort</code>	Insert a value into an already sorted array

Operation	Purpose
isSorted	Check whether the array is sorted
rearrange	Move negative values to the left and non-negative values to the right

These operations are different from basic insert/delete operations because they depend on the **position and order** of elements.

Part A: Insert into a Sorted Array

4. What Sorted Insertion Means

Sorted insertion means inserting a new value into an array that is already sorted.

Example:

```
A = [4, 8, 13, 16, 20, 25, 28, 33]
```

Suppose we want to insert:

```
18
```

The result should be:

```
[4, 8, 13, 16, 18, 20, 25, 28, 33]
```

The array should remain sorted after insertion.

5. Why Normal Append Is Not Enough

If we simply append `18` at the end, we get:

```
[4, 8, 13, 16, 20, 25, 28, 33, 18]
```

This is wrong because the array is no longer sorted.

So sorted insertion must find the correct position before placing the new value.

Simple idea:

Append places a value at the end. Sorted insertion places a value where it belongs.

6. Main Idea of Sorted Insertion

To insert a value into a sorted array:

1. Start from the last valid element.
2. Compare each element with the new value.
3. Shift larger elements one position to the right.
4. Stop when you find an element smaller than or equal to the new value.
5. Insert the new value after that element.
6. Increase `length`.

Simple memory:

Move bigger values right, then place the new value.

7. Why We Start from the Back

Suppose we want to insert `18` into:

```
[4, 8, 13, 16, 20, 25, 28, 33, _]
```

The values greater than `18` are:

```
20, 25, 28, 33
```

They must move one position right:

```
[4, 8, 13, 16, _, 20, 25, 28, 33]
```

Then `18` can be inserted into the empty position:

```
[4, 8, 13, 16, 18, 20, 25, 28, 33]
```

We start from the back because shifting from the front could overwrite values before they are moved.

This is the same safety idea you learned during the normal insert operation.

8. Sorted Insertion Code

```
void insertSort(struct Array *arr, int x) {
    if (arr->length == arr->size) {
        return;
    }

    int i = arr->length - 1;

    while (i >= 0 && arr->A[i] > x) {
        arr->A[i + 1] = arr->A[i];
        i--;
    }

    arr->A[i + 1] = x;
    arr->length++;
}
```

9. Explanation of Sorted Insertion Code

Capacity check

```
if (arr->length == arr->size) {
    return;
}
```

This checks whether the array is full.

If:

```
length == size
```

then there is no free space, so insertion should not happen.

This prevents writing outside the array.

Start from the last valid element

```
int i = arr->length - 1;
```

If `length = 5`, the last valid index is `4`.

So `i` starts at the final real element.

Shift larger elements right

```
while (i >= 0 && arr->A[i] > x) {  
    arr->A[i + 1] = arr->A[i];  
    i--;  
}
```

This loop continues while both conditions are true:

```
i >= 0
```

and

```
arr->A[i] > x
```

Meaning:

- `i >= 0` prevents going before the start of the array.
- `arr->A[i] > x` checks whether the current value is bigger than the new value.

If the current value is bigger, it must move right.

Insert at `i + 1`

```
arr->A[i + 1] = x;
```

Why `i + 1`?

Because the loop stops when either:

1. `i` moved before index `0`, or

2. `arr->A[i]` is not greater than `x`.

So the correct empty place is one position after `i`.

Increase length

```
arr->length++;
```

A new valid value has been added, so `length` increases by `1`.

10. Sorted Insertion Dry Run: Insert 18

Starting array:

```
A = [4, 8, 13, 16, 20, 25, 28, 33, _]  
length = 8  
x = 18
```

Initial:

```
i = length - 1 = 7
```

Trace:

Step	i	A[i]	Is <code>A[i] > 18</code> ?	Action
1	7	33	Yes	Move 33 to index 8
2	6	28	Yes	Move 28 to index 7
3	5	25	Yes	Move 25 to index 6
4	4	20	Yes	Move 20 to index 5
5	3	16	No	Stop loop
6	-	-	-	Put 18 at index <code>i + 1 = 4</code>

Final array:

```
[4, 8, 13, 16, 18, 20, 25, 28, 33]
```

Final length:

9

11. Sorted Insertion Dry Run: Insert 12

Starting array:

```
A = [2, 3, 5, 10, 15, _]  
length = 5  
x = 12
```

Initial:

```
i = 4  
A[i] = 15
```

Trace:

Step	i	A[i]	Is A[i] > 12?	Action
1	4	15	Yes	Shift 15 right
2	3	10	No	Stop
3	-	-	-	Place 12 at index 4

Final array:

[2, 3, 5, 10, 12, 15]

12. Best Case of Sorted Insertion

Best case happens when the new value is larger than every existing value.

Example:


```
A = [2, 4, 6, 8]
x = 10
```

Since 10 is already greater than the last value, no shifting is needed.

Result:

```
[2, 4, 6, 8, 10]
```

Time complexity:

```
O(1)
```

Why?

Because the loop does not shift any element.

13. Worst Case of Sorted Insertion

Worst case happens when the new value is smaller than every existing value.

Example:

```
A = [2, 4, 6, 8]
x = 1
```

Every element must shift right.

Result:

```
[1, 2, 4, 6, 8]
```

Time complexity:

```
O(N)
```

Why?

Because all `N` elements may need to move.

14. Space Complexity of Sorted Insertion

Sorted insertion uses only a few variables, such as:

`i, x`

It does not create another array.

So space complexity is:

$O(1)$

Part B: Check Whether an Array Is Sorted

15. What `isSorted` Means

`isSorted` checks whether the array is arranged in ascending order.

Ascending order means smaller values come before larger values.

Example of a sorted array:

`[2, 4, 6, 8, 10]`

Example of an unsorted array:

`[2, 4, 9, 6, 10]`

The problem is here:

`9 > 6`

Because a bigger value appears before a smaller value.

16. Main Idea of `isSorted`

To check if an array is sorted, compare every neighboring pair.

Check:

```
A[0] <= A[1]
A[1] <= A[2]
A[2] <= A[3]
...
```

If any pair is in the wrong order, the array is not sorted.

The wrong condition is:

```
A[i] > A[i + 1]
```

If that happens, return `0`.

If the loop finishes and no wrong pair is found, return `1`.

17. `isSorted` Code

```
int isSorted(struct Array arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        if (arr.A[i] > arr.A[i + 1]) {
            return 0;
        }
    }

    return 1;
}
```

This function returns:

Return value	Meaning
<code>1</code>	Array is sorted
<code>0</code>	Array is not sorted

18. Why the Loop Stops at `length - 1`

The loop condition is:

```
i < arr.length - 1
```

Why?

Because inside the loop, we compare:

```
arr.A[i] > arr.A[i + 1]
```

If `i` reached the last valid index, then `i + 1` would go outside the valid array.

Example:

```
length = 5  
last valid index = 4
```

If `i = 4`, then:

```
i + 1 = 5
```

But index `5` is not a valid element.

So the loop must stop before the last valid index.

19. `isSorted` Dry Run: Sorted Array

Array:

```
A = [2, 4, 6, 8, 10]  
length = 5
```

Trace:

Step	<code>i</code>	Compare	Result
1	0	<code>2 > 4</code> ?	No

Step	i	Compare	Result
2	1	4 > 6 ?	No
3	2	6 > 8 ?	No
4	3	8 > 10 ?	No

No wrong pair is found.

Return:

1

Meaning:

The array is sorted.

20. isSorted Dry Run: Unsorted Array

Array:

A = [2, 4, 9, 6, 10]
length = 5

Trace:

Step	i	Compare	Result
1	0	2 > 4 ?	No
2	1	4 > 9 ?	No
3	2	9 > 6 ?	Yes

The pair 9, 6 is in the wrong order.

Return:

0

Meaning:

The array is not sorted.

21. Best Case of `isSorted`

Best case happens when the first two elements are already out of order.

Example:

[8, 2, 4, 6]

The first comparison is:

$8 > 2$

This is true, so the function immediately returns `0`.

Time complexity:

$O(1)$

22. Worst Case of `isSorted`

Worst case happens when the array is fully sorted, or the only mistake is at the very end.

Example sorted array:

[2, 4, 6, 8, 10]

The function must check every neighboring pair.

Time complexity:

$O(N)$

23. Space Complexity of `isSorted`

`isSorted` uses only one loop variable:

`i`

It does not create another array.

Space complexity:

$O(1)$

Part C: Rearranging Negative and Positive Values

24. What Rearrangement Means

Rearrangement means moving negative values to the left side and positive values to the right side.

Example:

`[2, -3, 25, -15, -7, 10]`

After rearrangement:

`[-7, -3, -15, 25, 2, 10]`

The exact order may be different, but the goal is:

`negative values | non-negative values`

25. Important: Rearrangement Is Not Sorting

Rearrangement does not sort the array.

Example rearranged array:

```
[-7, -3, -15, 25, 2, 10]
```

This is rearranged correctly because all negative values are on the left and all positive values are on the right.

But it is not sorted.

A sorted version would look like:

```
[-15, -7, -3, 2, 10, 25]
```

So remember:

Rearrangement separates groups. Sorting arranges values in order.

26. How Rearrangement Works

The rearrangement operation uses two pointers:

```
i = 0  
j = length - 1
```

Pointer roles:

Pointer	Starts at	Moves	Looks for
<code>i</code>	Left side	Right	A positive or zero value on the left side
<code>j</code>	Right side	Left	A negative value on the right side

The goal is to find two misplaced values and swap them.

27. Pointer Movement Rules

Move `i`

```
while (arr->A[i] < 0) {  
    i++;  
}
```


This means:

Keep moving `i` while the value is already negative.

A negative value belongs on the left, so `i` can skip it.

Move `j`

```
while (arr->A[j] >= 0) {  
    j--;  
}
```

This means:

Keep moving `j` while the value is already non-negative.

A non-negative value belongs on the right, so `j` can skip it.

Swap when needed

```
if (i < j) {  
    swap(&arr->A[i], &arr->A[j]);  
}
```

If `i` is still before `j`, then:

- `A[i]` is a non-negative value on the left side.
- `A[j]` is a negative value on the right side.

So they are in the wrong places and should be swapped.

28. Swap Function Used by Rearrangement

```
void swap(int *x, int *y) {  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

This function swaps two values using their addresses.

Why use `temp`?

Because if we overwrite one value immediately, the original value is lost.

Example:

```
x = 2
y = -7
```

Step-by-step:

Step	Action	Result
1	<code>temp = x</code>	<code>temp = 2</code>
2	<code>x = y</code>	<code>x = -7</code>
3	<code>y = temp</code>	<code>y = 2</code>

Final result:

```
x = -7
y = 2
```

29. Rearrangement Code

A safer version checks `i < j` inside the inner loops:

```
void rearrange(struct Array *arr) {
    int i = 0;
    int j = arr->length - 1;

    while (i < j) {
        while (i < j && arr->A[i] < 0) {
            i++;
        }

        while (i < j && arr->A[j] >= 0) {
            j--;
        }

        if (i < j) {
            swap(&arr->A[i], &arr->A[j]);
        }
    }
}
```

```
    }  
  }  
}
```

30. Explanation of Rearrangement Code

Initialize two pointers

```
int i = 0;  
int j = arr->length - 1;
```

`i` starts at the first valid index.

`j` starts at the last valid index.

Main loop

```
while (i < j)
```

The operation continues while the two pointers have not met or crossed.

Once `i >= j`, the array has been rearranged enough.

Skip negatives on the left

```
while (i < j && arr->A[i] < 0) {  
    i++;  
}
```

If a value is negative and already on the left, it is fine. Move `i` forward.

Skip non-negatives on the right

```
while (i < j && arr->A[j] >= 0) {  
    j--;  
}
```

If a value is positive or zero and already on the right, it is fine. Move `j` backward.

Swap wrong values

```
if (i < j) {  
    swap(&arr->A[i], &arr->A[j]);  
}
```

This swaps a positive or zero value from the left side with a negative value from the right side.

31. Rearrangement Dry Run 1

Array:

A = [-4, 3, 8, -9, 10, -7, 5]

Goal:

negative values on the left, non-negative values on the right

Initial:

i = 0
j = 6

Trace:

Step	i stops at	j stops at	Action	Array
1	A[1] = 3	A[5] = -7	Swap	[-4, -7, 8, -9, 10, 3, 5]
2	A[2] = 8	A[3] = -9	Swap	[-4, -7, -9, 8, 10, 3, 5]
3	i = 3	j = 2	Stop	Done

Final array:

[-4, -7, -9, 8, 10, 3, 5]

All negative values are on the left.

32. Rearrangement Dry Run 2

Array:

```
A = [2, -3, 25, -15, -7, 10]
```

Initial:

```
i = 0  
j = 5
```

Trace:

Step	i stops at	j stops at	Action	Array
1	A[0] = 2	A[4] = -7	Swap	[-7, -3, 25, -15, 2, 10]
2	A[2] = 25	A[3] = -15	Swap	[-7, -3, -15, 25, 2, 10]
3	i = 3	j = 2	Stop	Done

Final array:

```
[-7, -3, -15, 25, 2, 10]
```

Again, the array is not sorted, but it is correctly rearranged.

33. Why Rearrangement Is In-Place

An operation is in-place when it modifies the original array without creating another full array.

`rearrange` is in-place because it uses only:

```
i, j, temp
```

It does not create a second array.

So the space complexity is:

$O(1)$

34. Complexity of Rearrangement

The two pointers scan the array from both sides.

Each pointer moves across the array only once.

So the time complexity is:

$O(N)$

Space complexity:

$O(1)$

because only a few variables are used.

Part D: Pass-by-Value vs Pass-by-Address in Phase 7

35. Which Functions Modify the Array?

Function	Changes array?	Passing style
<code>insertSort</code>	Yes	Pass by address
<code>isSorted</code>	No	Pass by value
<code>rearrange</code>	Yes	Pass by address

36. Why `insertSort` Uses a Pointer

`insertSort` changes the real array because it:

1. Shifts elements.
2. Inserts a new value.

3. Increases `length`.

So it must receive:

```
struct Array *arr
```

Inside the function, use arrow notation:

```
arr->A[i]  
arr->length  
arr->size
```

37. Why `isSorted` Uses Pass-by-Value

`isSorted` only reads the array.

It does not change:

- values in `A`
- `size`
- `length`

So it can receive:

```
struct Array arr
```

Inside the function, use dot notation:

```
arr.A[i]  
arr.length
```

38. Why `rearrange` Uses a Pointer

`rearrange` changes the actual order of the array by swapping values.

So it must receive:

```
struct Array *arr
```

Inside the function, use arrow notation:

```
arr->A[i]  
arr->length
```

Simple memory:

Read only? Pass by value. Modify the real array? Pass by address.

Part E: Complete Phase 7 Program

39. Complete C Program

```
#include <stdio.h>  
  
struct Array {  
    int A[20];  
    int size;  
    int length;  
};  
  
void display(struct Array arr) {  
    for (int i = 0; i < arr.length; i++) {  
        printf("%d ", arr.A[i]);  
    }  
    printf("\n");  
}  
  
void swap(int *x, int *y) {  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}  
  
void insertSort(struct Array *arr, int x) {  
    if (arr->length == arr->size) {  
        return;  
    }  
  
    int i = arr->length - 1;
```



```

        while (i >= 0 && arr->A[i] > x) {
            arr->A[i + 1] = arr->A[i];
            i--;
        }

        arr->A[i + 1] = x;
        arr->length++;
    }

    int isSorted(struct Array arr) {
        for (int i = 0; i < arr.length - 1; i++) {
            if (arr.A[i] > arr.A[i + 1]) {
                return 0;
            }
        }
        return 1;
    }

    void rearrange(struct Array *arr) {
        int i = 0;
        int j = arr->length - 1;

        while (i < j) {
            while (i < j && arr->A[i] < 0) {
                i++;
            }

            while (i < j && arr->A[j] >= 0) {
                j--;
            }

            if (i < j) {
                swap(&arr->A[i], &arr->A[j]);
            }
        }
    }

    int main() {
        struct Array arr1 = {{2, 3, 5, 10, 15}, 20, 5};

        printf("Original sorted array:\n");
        display(arr1);

        insertSort(&arr1, 12);
        printf("After inserting 12:\n");
        display(arr1);
    }

```

```

    if (isSorted(arr1)) {
        printf("Array is sorted.\n");
    } else {
        printf("Array is not sorted.\n");
    }

    struct Array arr2 = {{2, -3, 25, -15, -7, 10}, 20, 6};

    printf("\nOriginal mixed array:\n");
    display(arr2);

    rearrange(&arr2);
    printf("After rearranging negatives and positives:\n");
    display(arr2);

    return 0;
}

```

40. Program Walkthrough

Starting sorted array

[2, 3, 5, 10, 15]

Call:

insertSort(&arr1, 12);

The function shifts 15 right and inserts 12 before it.

Result:

[2, 3, 5, 10, 12, 15]

Then:

isSorted(arr1)

returns:

1

because the array is sorted.

Starting mixed array

[2, -3, 25, -15, -7, 10]

Call:

```
rearrange(&arr2);
```

One possible result:

[-7, -3, -15, 25, 2, 10]

This means all negative values are on the left and non-negative values are on the right.

41. Expected Output

One possible output is:

Original sorted array:

2 3 5 10 15

After inserting 12:

2 3 5 10 12 15

Array is sorted.

Original mixed array:

2 -3 25 -15 -7 10

After rearranging negatives and positives:

-7 -3 -15 25 2 10

The rearranged output can vary depending on the swap order, but the rule must hold:

negative values first, non-negative values after

Part F: Complexity Summary

42. Phase 7 Complexity Table

Let:

```
N = arr.length
```

Operation	Best Time	Worst Time	Space	Reason
<code>insertSort</code>	<code>O(1)</code>	<code>O(N)</code>	<code>O(1)</code>	May shift no elements or all elements
<code>isSorted</code>	<code>O(1)</code>	<code>O(N)</code>	<code>O(1)</code>	May stop early or check all neighboring pairs
<code>rearrange</code>	<code>O(N)</code>	<code>O(N)</code>	<code>O(1)</code>	Two pointers scan the array

43. Why `insertSort` Can Be `O(1)` or `O(N)`

If the new value belongs at the end, no shifting is needed.

Example:

```
[2, 4, 6, 8]  
insert 10
```

Time:

```
O(1)
```

If the new value belongs at the beginning, every value shifts right.

Example:

```
[2, 4, 6, 8]  
insert 1
```

Time:

$O(N)$

44. Why `isSorted` Can Be $O(1)$ or $O(N)$

If the first pair is already wrong:

[9, 2, 4, 6]

The function returns immediately.

Time:

$O(1)$

If the array is sorted:

[2, 4, 6, 8]

The function must check every neighboring pair.

Time:

$O(N)$

45. Why `rearrange` Is Always $O(N)$

The two pointers move across the array from opposite sides.

Even though they move in different directions, together they still scan the array.

So the total work grows with the number of valid elements.

Time:

$O(N)$

Part G: Common Beginner Mistakes

46. Mistake 1: Using Sorted Insertion on an Unsorted Array

Wrong situation:

```
A = [10, 3, 7, 20]
insert 8
```

This array is not sorted before insertion.

`insertSort` assumes the existing array is already sorted.

Correct rule:

Use sorted insertion only when the array is already sorted.

If the array is unsorted, first sort it or use a normal insert operation.

47. Mistake 2: Forgetting the Capacity Check

Wrong code:

```
void insertSort(struct Array *arr, int x) {
    int i = arr->length - 1;
    // shifting code...
}
```

Problem:

If the array is already full, the function may write outside the array.

Correct code:

```
if (arr->length == arr->size) {
    return;
}
```

48. Mistake 3: Using `i > 0` Instead of `i >= 0`

Wrong:

```
while (i > 0 && arr->A[i] > x)
```

Problem:

This does not check index `0`.

If the new value should be inserted before the first element, the code may fail.

Correct:

```
while (i >= 0 && arr->A[i] > x)
```

49. Mistake 4: Forgetting `i + 1` Placement

Wrong idea:

```
arr->A[i] = x;
```

Correct:

```
arr->A[i + 1] = x;
```

Why?

After the loop stops, `i` points to the value before the insertion position.

So the new value belongs at:

```
i + 1
```

50. Mistake 5: Looping Too Far in `isSorted`

Wrong:

```
for (int i = 0; i < arr.length; i++) {  
    if (arr.A[i] > arr.A[i + 1]) {  
        return 0;  
    }  
}
```

Problem:

When `i` reaches the last valid index, `i + 1` goes outside the valid array.

Correct:

```
for (int i = 0; i < arr.length - 1; i++)
```

51. Mistake 6: Thinking Rearrangement Sorts the Array

Wrong thinking:

```
rearrange = sort
```

Correct thinking:

```
rearrange = separate negatives and non-negatives
```

Example:

```
[-7, -3, -15, 25, 2, 10]
```

This is rearranged correctly, but it is not sorted.

52. Mistake 7: Forgetting Pointer Boundaries in Rearrangement

A safer version checks `i < j` in the inner loops:


```

while (i < j && arr->A[i] < 0) {
    i++;
}

while (i < j && arr->A[j] >= 0) {
    j--;
}

```

This prevents the pointers from crossing too far while scanning.

Part H: Phase 7 Final Summary

53. Main Ideas

Phase 7 teaches:

1. How to insert into a sorted array.
2. How to check if an array is sorted.
3. How to rearrange negative and positive values.

54. Operation Summary

Operation	Main logic	Changes array?	Time
<code>insertSort</code>	Shift larger values right, then insert	Yes	Best <code>O(1)</code> , worst <code>O(N)</code>
<code>isSorted</code>	Compare every neighboring pair	No	Best <code>O(1)</code> , worst <code>O(N)</code>
<code>rearrange</code>	Use two pointers and swap wrong-side values	Yes	<code>O(N)</code>

55. One-Line Memory for Each Operation

`insertSort`: move bigger values right, then insert.

isSorted: check if any neighbor pair is in the wrong order.

rearrange: negatives left, non-negatives right.

56. Checkpoint Questions

You are ready for the next phase when you can answer these:

1. Why must the array already be sorted before using `insertSort`?
2. Why does sorted insertion start from the last valid element?
3. Why do we check `arr->length == arr->size` before insertion?
4. What are the two conditions in the sorted insertion `while` loop?
5. Why is the final insertion position `i + 1`?
6. What does `isSorted` return for a sorted array?
7. What does `isSorted` return for an unsorted array?
8. Why does `isSorted` loop only while `i < length - 1`?
9. What is the purpose of pointer `i` in `rearrange`?
10. What is the purpose of pointer `j` in `rearrange`?
11. Why is `rearrange` not the same as sorting?
12. Why is `rearrange` called an in-place operation?
13. Which Phase 7 functions use pass-by-address?
14. Which Phase 7 function can use pass-by-value?
15. What is the worst-case time complexity of sorted insertion?
16. What is the worst-case time complexity of checking sorted order?
17. What is the time complexity of rearrangement?

57. Final One-Line Summary

Phase 7 is about working with ordered array logic: inserting into sorted arrays, checking sorted order, and using two pointers to separate negative and non-negative values in-place.